

Analysis: De-RNG-ed

Finding the next integer in the sequence means figuring out the values of **A**, **B** and **P** that fit the given numbers and using them to generate the next number.

This problem has a few special cases. In all of them, it is important to note that the value of **P** must always be larger than every element of the "randomly" generated sequence. (8 can never be a remainder after dividing by 7.) Also, the problem statement requires that **P** be no larger than 10^9 . Let's call all primes that satisfy both of these bounds "valid primes". Now, let's look at the special cases.

K = 1

First of all, when **K** is 1, the answer is always "I don't know." This is because we can pick any valid prime **P**, set **A** to 0 and **B** to 0 or 1. This will give us two different answers.

K = 2 and the two sequence elements are the same

In this case, the answer is unique because the next element of the sequence depends only on the current element. If two consecutive elements are the same, then the entire sequence consists of a single repeated number.

K > 2 and all the sequence elements are the same

Similarly, the next element must be the same as all other elements.

K = 2 and the two sequence elements are different

Here, the answer is always "I don't know." To see that, pick any valid prime and consider the cases **A**=0 and **A**=1. If we call the first element of the sequence **x** and the second element **y**, we can express **y** as a function of **x**, **A**, **B** and **P**: $y = (A \cdot x + B) \% P$. In both cases, we can solve this equation for **B**. The next element, **z** is then $z = (A \cdot y + B) \% P$, and it must be different in the two cases (**A**=0 and **A**=1) as long as **x** is different from **y**.

K = 3

We are going to brute force all valid primes and solve for **A** and **B**. We will then use these values to generate the next element of the sequence. If all the values we get this way are the same, then the answer is unique. If we get different valid answers, then the answer is "I don't know."

Let's call the 3 elements **x**, **y** and **z**. By writing **y** as a function of **x** and **z** as a function of **y** and subtracting **y** from **z**, we get

$$z - y = (A \cdot y + B) - (A \cdot x + B) = A \cdot (y - x) \pmod{P}.$$

We have already dealt with the case when **x** equals **y**, so we can assume that **x** and **y** are different, so we can divide by their difference. This lets us solve for **A**.

$$A = (z - y) \cdot (y - x)^{-1} \pmod{P}.$$

Computing the inverse of $(y - x)$ can be done using the [Euclidean algorithm](#), which runs in $O(\log(P))$ time.

Once we have **A**, solving for **B** is easy:
B = **y** - **A*****x**.

The answer is then **A*****z** + **B**.

K > 3

In this case, we brute force **P**, use the first 3 elements of the sequence to solve for **A** and **B**, and check whether the remaining elements fit the sequence generated with these parameters.

Analysis: Different Sum

The small input

The solution for the small input of this problem was quite straightforward. One could iterate over all possible ways to partition N into a sum of positive integers, and verify that each column has distinct digits.

Since there are 190569292 partitions of 100 into a sum of positive integers, this algorithm might not run fast enough. However, we can optimize it with an easy observation: the summands must be distinct. That brings the total number of partitions of 100 down to just 444793, which is small enough for our needs.

But if you want to cut the search space down even further, you can use *backtracking*. This is a general technique that works as follows in this problem: as you're generating the partition, you can check if there's a column that has two equal digits after adding each number, not just in the end. That way, many bad partitions get filtered out early and you have even less possibilities to check.

The large input

In order to approach the large input, we need to rotate ourselves 90 degrees. In the above solution, we've generated our cryptarithm from top to bottom. Now, we will generate it from right to left.

First, we check all possibilities for the digits in the rightmost (least significant) column such that the last digit of their sum matches the required one. Then, we continue with the digits for the next-to-rightmost column, and so on.

Suppose we have already filled a few rightmost columns. We can note that the things that are relevant for us now is the value V of carry from the already filled columns to the next one, the amount K of summands in the column that was just filled, and the boolean flag F indicating whether there has been a zero in the column that was just filled (this flag is important since it affects whether we can terminate the corresponding number now). When we know the values of V , K and F , the actual digits in the already filled columns don't affect the further execution of the algorithm.

This observation logically leads us to the following *Dynamic Programming* solution: let's calculate $\text{Count}[i, V, K, F]$ which is defined as the number of ways to place the digits in the last i columns in such a way that the sum in those columns matches N , there's a carry of V , the number of summands that have at least i digits is K , and F is 1 when there's a summand that starts with zero, 0 otherwise.

In order to calculate $\text{Count}[i+1, \dots]$ given $\text{Count}[i, \dots]$, we need to consider all possible ways to place up to K digits in the $i+1$ -th rightmost column. K is up to B (since all digits in one column are different, the number of summands doesn't exceed the number of different digits), which can be up to 100 in the large input. From the first glance, this gives us at least 100! (factorial of 100) possibilities, rendering our idea still useless.

But now's when another Dynamic Programming idea comes into play! One can notice that we don't need to know exactly all digits of the $i+1$ -th column. The important thing for us is the amount of those digits, the sum of those digits, and whether one of them is zero. When we know those, we can multiply our answer by an appropriate number (which will be a product of

binomial coefficients and factorials) to account for various ways to attach those digits to the already formed numbers in the first i columns.

So we run a separate Dynamic Programming that calculates **Count2**[K , S , F] which is defined as the number of ways to place K distinct digits in a column such that their sum is S and F denotes whether one of them is zero. K is up to B , S is $O(B^2)$, meaning we get $O(B^3)$ states, which is small enough.

The main Dynamic Programming has $O(B^2 \cdot \text{number_of_digits})$ states, and using the **Count2** table each state can be processed in $O(B^2)$ by looking at the number of digits in the $i+1$ -th column and the carry to the $i+2$ -th column (the required sum in the $i+1$ -th column is uniquely determined by the carry to it, the carry from it, and the corresponding digit of N). The total runtime of this solution is thus $O(B^4 \cdot \text{number_of_digits})$.

Analysis: Fence

The basic scenario here is very similar to the traditional change-making problem, except that the input can be (and actually is guaranteed to be) very large. A condition on the minimum size of the input is very unusual for a programming contest problem, and we didn't add it just for fun. Our solution really, truly does require that the fence length be at least 10^{10} .

Small Input

Before getting into the real solution though, let's discuss a simpler approach that at least solves the small input. Let's suppose the longest board is of length $A \leq 100$. The key idea is that we should never use more than A boards of any size less than A . If we did, we could replace A of those boards with a smaller number of length- A boards. And that would of course be a better solution.

In particular, this means the total length of all shorter boards is at most $N * A * A \leq 10000000$. Using a breadth-first search, we can find the optimal way of choosing these boards to get each length in that range. The cost of completing the fence using length- A boards can then be computed with a simple division.

By the way, you can actually replace $N * A * A$ with just $A * A$ in the above solution. Hopefully you will see why after reading the rest of the solution!

Large Input

The small-input solution does not actually take advantage of the minimum length of the fence. So the big question is: how could we possibly do that?

Well, the previous solution offers a bit of a hint. For a really long fence, it makes sense that in the end, we are going to want to make heavy use of the longest board just to cover up as much length as possible. So let's suppose the longest board has length A , and that L is equal to $p * A + q$ for integers p, q with $q < A$. (Note that the problem statement guarantees $p \geq A$.) Then we need to do one of the following things:

- Use a number $T_{0,q}$ of shorter boards to create a fence of length $0 * A + q$, then use p boards of length A to cover the rest.
- Use a number $T_{1,q}$ of shorter boards to create a fence of length $1 * A + q$, then use $p-1$ boards of length A to cover the rest.
- ...
- Use a number $T_{p,q}$ of shorter boards to create a fence of length $p * A + q$, then use 0 boards of length A to cover the rest.

So we need to calculate $p + S_{p,q}$ where $S_{p,q}$ is defined to be $\min(T_{0,q} - 0, T_{1,q} - 1, \dots, T_{p,q} - p)$. Intuitively, $S_{p,q}$ can be thought of as measuring the minimal number of boards required to get a fence length of $q \bmod A$, subject to two modifications:

- Every time the length increases by A , it means one less max-length board in the future, so you can subtract one from the total count.
- The total length is not allowed to go over $p * A + q$.

And now, we can make concrete how the condition that L is very large simplifies things:

Lemma: The second condition in the definition of $S_{p,q}$ is unnecessary.

Proof: We claim that $T_{i,q} - i$ is minimized when $i \leq p$, which will prove the lemma. So let's consider a minimal $T_{i,q} - i$. Then we have a set of boards $b_1, b_2, \dots, b_m$ making a length of $i \cdot A + q$. If $i > p$, then $m > p \geq A$. But then, the set $\{b_1, b_1 + b_2, \dots, b_1 + b_2 + \dots + b_m\}$ contains at least $A+1$ numbers, so two of these numbers are congruent modulo A . Subtracting them, we can find a non-empty subset of $\{b_1, b_2, \dots, b_m\}$ whose sum is a multiple of A . Therefore, we can replace that subset with boards of length A to get a strictly better solution, implying $T_{i,q} - i$ could not have been optimal in the first place!

Okay, that's all very nice, but what's the solution? Well the previous lemma implies we need to calculate the minimum number of boards required to get a fence of length $q \bmod A$, subject to the fact that each time the total goes up by A , we will need one fewer board in the future. (For shorter fences, this approach just does not work. Our algorithm would make a very long fence with length correct modulo A , and then try to subtract length- A boards, which of course is not allowed!)

Anyway, once the problem has been reduced in this way, it can be done pretty straightforwardly with a breadth-first search. Our graph has one vertex for each residue modulo A . From each vertex, we add an edge for each possible board length. If adding that board involves wrapping past A , then it has weight 0. Otherwise, it has weight 1. So the final algorithm is: calculate the minimum distance in this graph to vertex q to get $S_{p,q}$, and finally add p .

Analysis: Hot Dog Proliferation

Background

This analysis will have nothing to do with hot dogs. Instead of a long street with billions of corners, let's think of the line of integers; instead of vendors, let's think of chips -- after all, chips are much easier to maneuver than real people with hot dog stands!

We denote the number of total chips by n . Also, let's call a configuration *stable* if no two chips occupy the same integer point.

If you play around with the game for a while or if you have good (and brave!) intuition, you might realize that the problem statement is a little misleading. It turns out that no matter which move you do at each step, the final configuration, as well as the total number of moves you need to perform, will always be the same.

Indeed, this is a famous theorem for "chip-firing games", and our scenario is a special kind of chip-firing game. Intuitively, the reason why your choices don't matter is that (a) if you ignore a move now, you will still have to do it later, and (b) one move will not change the effect of another move down the line. This means that while you can control the *order* of moves, you will always do the same set of moves in the end, and they will always have the same effect.

This observation is enough to solve the small input. Just keep doing moves until the configuration stabilizes, and count how long it took. For the large input though, more insight is required. A configuration might require over 10^{13} moves to stabilize, so simulating them one at a time is out of the question. The obvious optimization is to do several moves at once for very large piles. Surprisingly however, this does not help very much.

There are a few different ways to proceed, and we will discuss two of them.

Preliminary Observations

One very useful way of understanding this game is in terms of *invariants*. The first of these is pretty obvious, but the other requires either some special insight or some experience to see. In each move, we take two chips at some position x and send them to positions $x-1$ and $x+1$. Notice that:

$$\begin{aligned}(x-1) + (x+1) &= x + x \\ (x-1)^2 + (x+1)^2 &= x^2 + x^2 + 2.\end{aligned}$$

This immediately leads to the following two observations:

Observation 1. The sum of the positions of all the chips never changes.

Observation 2. The sum of the squared positions of all the chips increases by 2 during each move.

So how do we use these observations? They aren't *necessary*, but they will have their uses as you will see. The former one will help us quickly construct a configuration with certain known properties from the initial configuration (more on this later). And with the latter observation, computing the number of steps becomes the same task as constructing the final configuration.

For example, using Observation 2, we can easily estimate that the number of steps could be on the order of n^3 , thereby verifying that straightforward simulation really is hopeless.

Adding one chip

One good approach is to add chips one at a time, at each step doing enough moves to completely stabilize the configuration. The question is: how do we do this last part efficiently? So let's consider adding a chip to a stable configuration.

If the new chip arrives at a position where there was no chip before, we are done. Otherwise, it lands on a segment, and the picture looks something like this:

*

?????????.*****.?????????

The two "."s represents empty positions. If you play around with a couple examples, you should be able to see that the ending result will always be two segments, one starting from the position of the left "." in the picture, and the other ending at the position of the right ".". We might also view the result as a single segment with a hole. Furthermore, you might also realize that if there were A points to the left of our new chip in the original configuration, and B points to the right, then the two new segments will have lengths $B+1$ and $A+1$ respectively, and the total number of moves required will be $(A+1) * (B+1)$.

We could also have computed the position of the hole using Observation 1. The sum of the positions in the initial configuration is $(1+2+\dots+15)+4$, and we know in the new configuration that the sum is $(0+1+\dots+16)-H$, where H is the position of the hole. Therefore, H must be 12. The final picture is

?????????*****.*****?????????

We could then use Observation 2 to easily determine how many moves were required to get here.

So here is one possible solution to the problem. Add the chips one by one. At each stage, we have up to n disjoint segments. If the new chip lands on an unoccupied position, it forms a segment unto itself; otherwise, it transforms one segment into two as described above. In either case, the new segments might touch the ones to their left and/or right, and we merge them if that happens.

All that's left is to figure out how to store these segments in your program. If you are clever, you might realize that if we add the chips from left to right, then each new chip will always be on or next to one of the last two segments. You could then use a stack to store all the segments -- all the operations will be on the top two elements of the stack. This approach gives an $O(n)$ solution. If you missed this last insight, you could also use a binary search tree (e.g. an STL set) to get an $O(n \log n)$ solution.

Adding one pile

In our problem, we have C piles of chips, and usually C is much smaller than n . We now sketch a lightning-fast solution that runs in $O(C)$ time. This level of insight is not necessary to solve the problem, but it's still pretty interesting. As you will see, it is essential to understand the details of the above $O(n)$ solution.

Instead of adding one chip at a time, we will try to process all the chips from a single position at the same time.

First let's resolve the case when there is only one pile of n chips at position x . By symmetry and the discussions in the previous section, it is easy to see that the stable configuration is a segment centered at x if n is odd; and a segment centered at x with a hole in the center if n is even.

Let's define an *H-segment* to be a segment with a hole. It is a tuple (x, y, z) , where $x < y \leq z$, representing a segment of chips from position x to position z , inclusive, but with position y empty. Note that, when $y = z$, the hole is at the very end, and it is actually a normal segment.

Our solution adds the piles one by one. And we keep a stack of existing H-segments from the left to the right. When a new pile comes, it is transformed into a new H-segment. If the H-segment does not overlap with any existing H-segments, we are done. Otherwise, it overlaps with the topmost H-segment in the stack; that is, it creates some positions with two chips. But using the observations from the last section, we know that if we resolve the conflicts one at a time, we will always have at most one hole. That means the result will be another H-segment. If the new one overlaps with the current top H-segment in the stack, we continue with the same resolving process. We do this until the stack is empty, or the H-segment is disjoint from the top of the stack. Then we push the new one and proceed to the next pile.

It remains only to explain how to compute a new H-segment quickly. And the answer is: just use Observation 1 again! When resolving two H-segments, we know S -- the sum of the positions in them; we also know the total number of chips K , so (remember the hole), $z = x + K$. We need to decide the start position x . Depending on y , the sum S satisfies

$$K(2x + K - 1) / 2 \leq S < K(2x + K + 1) / 2$$

There is a unique x satisfying this, and it can be solved in constant time. We can then find y exactly like we did in the $O(n)$ solution.

More Information

If you liked this problem, you might also enjoy reading the following classical paper on chip-firing games:

- Anders Björner, László Lovász, and Peter Shor *Chip-firing games on graphs*. European Journal of Combinatorics, Volume 12 , Issue 4 (July 1991).